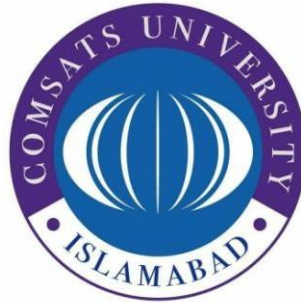


COMSATS University Islamabad

Attock Campus



Department Of Computer Science

<i>Course</i>	<i>Information Security</i>
<i>Instructor</i>	<i>Ms. Ambreen Gul</i>
<i>Lab Assignment</i>	<i>03</i>

Student Details

<i>Registration No.</i>	<i>Name</i>
<i>FA24-BSE-008</i>	<i>Muhammad Afnan</i>

```

1  import hashlib
2  import time
3
4
5  # TASK 1: BASIC BLOCKCHAIN IMPLEMENTATION
6
7  class Block:
8      def __init__(self, index, timestamp, data, previous_hash):
9          self.index = index
10         self.timestamp = timestamp
11         self.data = data
12         self.previous_hash = previous_hash
13         self.hash = self.calculate_hash()
14
15         # Function to calculate hash using SHA256
16         def calculate_hash(self):
17             block_string = str(self.index) + str(self.timestamp) + str(self.data)
18             return hashlib.sha256(block_string.encode()).hexdigest()
19
20
21  class Blockchain:
22      def __init__(self):
23          self.chain = [self.create_genesis_block()]
24
25          # Genesis block (first block)
26          def create_genesis_block(self):
27              return Block(0, time.ctime(), "Genesis Block", "0")
28
29          # Get last block
30          def get_latest_block(self):
31              return self.chain[-1]
32
33          # Add new block
34          def add_block(self, data):
35              latest_block = self.get_latest_block()
36              new_index = latest_block.index + 1
37              new_timestamp = time.ctime()
38              new_previous_hash = latest_block.hash
39
40              new_block = Block(new_index, new_timestamp, data, new_previous_hash)
41              self.chain.append(new_block)
42
43          # Display blockchain
44          def display_chain(self):
45              for block in self.chain:
46                  print("=====")
47                  print(f"Index: {block.index}")
48                  print(f"Timestamp: {block.timestamp}")
49                  print(f>Data: {block.data}")
50                  print(f"Previous Hash: {block.previous_hash}")
51                  print(f"Hash: {block.hash}")
52                  print("=====")
53
54          # Validate blockchain
55          def is_chain_valid(self):
56              for i in range(1, len(self.chain)):
57                  current_block = self.chain[i]
58                  previous_block = self.chain[i - 1]
59
60                  # Check if current hash is correct
61                  if current_block.hash != current_block.calculate_hash():
62                      return False

```

```

63
64         # Check if previous hash matches
65         if current_block.previous_hash != previous_block.hash:
66             return False
67
68     return True
69
70
71 # TASK 2: ECC BASICS (POINT ADDITION, DOUBLING, KEY GEN, ENCRYPTI
72
73 # Elliptic Curve:  $y^2 = x^3 + ax + b \pmod{p}$ 
74 a = 2
75 b = 3
76 p = 97 # small prime field
77
78
79 # Function to find modular inverse
80 def mod_inverse(k, p):
81     return pow(k, p - 2, p)
82
83
84 # Point addition function
85 def point_add(P, Q):
86     if P is None:
87         return Q
88     if Q is None:
89         return P
90
91     x1, y1 = P
92     x2, y2 = Q
93
94     # If points are inverse of each other
95     if x1 == x2 and (y1 + y2) % p == 0:
96         return None
97
98     # If points are same, do doubling
99     if P == Q:
100         return point_double(P)
101
102     # Slope formula for addition
103     m = ((y2 - y1) * mod_inverse((x2 - x1) % p, p)) % p
104
105     x3 = (m * m - x1 - x2) % p
106     y3 = (m * (x1 - x3) - y1) % p
107
108     return (x3, y3)
109
110
111 # Point doubling function
112 def point_double(P):
113     if P is None:
114         return None
115
116     x1, y1 = P
117
118     # slope formula for doubling
119     m = ((3 * x1 * x1 + a) * mod_inverse((2 * y1) % p, p)) % p
120
121     x3 = (m * m - 2 * x1) % p
122     y3 = (m * (x1 - x3) - y1) % p
123

```

```

124         return (x3, y3)
125     Click to add a breakpoint
126
127     # Scalar multiplication (kP)
128     def scalar_multiply(k, P):
129         result = None
130         temp = P
131
132         while k > 0:
133             if k % 2 == 1:
134                 result = point_add(result, temp)
135                 temp = point_double(temp)
136             k //= 2
137
138         return result
139
140     def ecc_encrypt(message, G, public_key):
141         k = 3 # random small number (normally random)
142         C1 = scalar_multiply(k, G)
143         shared_secret = scalar_multiply(k, public_key)
144
145         # if shared secret becomes None, encryption fails
146         if shared_secret is None:
147             raise ValueError("Shared secret became Point at Infinity. Choose")
148
149         # Encrypt message as integer (simple method)
150         C2 = (message + shared_secret[0]) % p
151         return C1, C2
152
153

```

```

154     # Simplified EC-ElGamal decryption
155     def ecc_decrypt(C1, C2, private_key):
156         shared_secret = scalar_multiply(private_key, C1)
157         message = (C2 - shared_secret[0]) % p
158         return message
159
160
161     # MAIN PROGRAM EXECUTION
162
163     print("\n===== TASK 1: BLOCKCHAIN =====\n")
164
165     # Create blockchain
166     my_blockchain = Blockchain()
167
168     # Add blocks
169     my_blockchain.add_block("Block 1: Afnan paid 100 PKR")
170     my_blockchain.add_block("Block 2: Afnan paid 250 PKR")
171     my_blockchain.add_block("Block 3: Afnan paid 500 PKR")
172     my_blockchain.add_block("Block 4: Afnan paid 1000 PKR")
173
174     print("Original Blockchain:")
175     my_blockchain.display_chain()
176
177     print("Is Blockchain Valid?", my_blockchain.is_chain_valid())
178
179     # Tampering with block data
180     print("\n--- Modifying Block 2 Data (Tampering) ---\n")
181     my_blockchain.chain[2].data = "Block 2: Afnan paid 999999 PKR"
182     my_blockchain.chain[2].hash = my_blockchain.chain[2].calculate_hash()
183

```

```

185 print("Is Blockchain Valid After Tampering?", my_blockchain.is_ch
186
187
188 print("\n===== TASK 2: ECC IMPLEMENTATION =====")
189
190 # Generator point (must lie on curve)
191 G = (3, 6)
192
193 print("Elliptic Curve:  $y^2 = x^3 + 2x + 3 \pmod{97}$ ")
194 print("Generator Point G =", G)
195
196 # Private Key
197 private_key = 7
198
199 # Public Key = private_key * G
200 public_key = scalar_multiply(private_key, G)
201
202 print("\nPrivate Key:", private_key)
203 print("Public Key:", public_key)
204
205 # Message to encrypt (numeric message)
206 message = 20
207 print("\nOriginal Message:", message)
208
209 # Encrypt
210 C1, C2 = ecc_encrypt(message, G, public_key)
211 print("\nEncrypted Cipher:")
212 print("C1 =", C1)
213 print("C2 =", C2)
214
215 # Decrypt
216 decrypted_message = ecc_decrypt(C1, C2, private_key)
217 print("\nDecrypted Message:", decrypted_message)
218
219 if decrypted_message == message:
220     print("\nECC Encryption/Decryption Successful!")
221 else:
222     print("\nECC Encryption/Decryption Failed!")
223

```

OUTPUT:

```
===== TASK 1: BLOCKCHAIN =====

Original Blockchain:
=====
Index: 0
Timestamp: Sat May  2 13:23:00 2026
Data: Genesis Block
Previous Hash: 0
Hash: 10de5846b5b8475116ffd90a7a4f6222a2b9901fde6d429c9984f422f49bc544
=====
Index: 1
Timestamp: Sat May  2 13:23:00 2026
Data: Block 1: Afnan paid 100 PKR
Previous Hash: 10de5846b5b8475116ffd90a7a4f6222a2b9901fde6d429c9984f422f49bc544
Hash: e1adb1eacc3461a807381e8ce6a9620976cd70f8eaf96b43e006b2282a453c56
=====
Index: 2
Timestamp: Sat May  2 13:23:00 2026
Data: Block 2: Afnan paid 250 PKR
Previous Hash: e1adb1eacc3461a807381e8ce6a9620976cd70f8eaf96b43e006b2282a453c56
Hash: a543406a2742e6ad4e3dd4ca572aeb67fce11c6c37767edba2dda026f884b9fa
=====
Index: 3
Timestamp: Sat May  2 13:23:00 2026
Data: Block 3: Afnan paid 500 PKR
Previous Hash: a543406a2742e6ad4e3dd4ca572aeb67fce11c6c37767edba2dda026f884b9fa
Hash: f1e1fc1240a4dabb7533b3a74a7b8aad42cfeda1fb11d3f958787c6bdd71d733
=====
Index: 4
Timestamp: Sat May  2 13:23:00 2026
Data: Block 4: Afnan paid 1000 PKR
```

```
Data: Block 4: Afnan paid 1000 PKR
Previous Hash: f1e1fc1240a4dabb7533b3a74a7b8aad42cfeda1fb11d3f958787c6bdd71d733
Hash: 49b7c4d2996a5f2e51a9417a240c2eb27edf6b51860db687fc3283b4cb7ce9fd
=====
Is Blockchain Valid? True
```

--- Modifying Block 2 Data (Tampering) ---

```
Blockchain After Modification:
=====
Index: 0
Timestamp: Sat May  2 13:23:00 2026
Data: Genesis Block
Previous Hash: 0
Hash: 10de5846b5b8475116ffd90a7a4f6222a2b9901fde6d429c9984f422f49bc544
=====
Index: 1
Timestamp: Sat May  2 13:23:00 2026
Data: Block 1: Afnan paid 100 PKR
Previous Hash: 10de5846b5b8475116ffd90a7a4f6222a2b9901fde6d429c9984f422f49bc544
Hash: e1adb1eacc3461a807381e8ce6a9620976cd70f8eaf96b43e006b2282a453c56
=====
Index: 2
Timestamp: Sat May  2 13:23:00 2026
Data: Block 2: Afnan paid 999999 PKR
Previous Hash: e1adb1eacc3461a807381e8ce6a9620976cd70f8eaf96b43e006b2282a453c56
Hash: 7ad143f7fbb2e84aa962547d1e9df0e0139079b5c3949092ec112308cf972856
=====
Index: 3
Timestamp: Sat May  2 13:23:00 2026
Data: Block 3: Afnan paid 500 PKR
```

```
Data: Block 3: Afnan paid 500 PKR
Previous Hash: a543406a2742e6ad4e3dd4ca572aeb67fce11c6c37767edba2dda026f884b9fa
Hash: f1e1fc1240a4dabb7533b3a74a7b8aad42cfeda1fb11d3f958787c6bdd71d733
=====
Index: 4
Timestamp: Sat May  2 13:23:00 2026
Data: Block 4: Afnan paid 1000 PKR
Previous Hash: f1e1fc1240a4dabb7533b3a74a7b8aad42cfeda1fb11d3f958787c6bdd71d733
Hash: 49b7c4d2996a5f2e51a9417a240c2eb27edf6b51860db687fc3283b4cb7ce9fd
=====
```

Is Blockchain Valid After Tampering? False

===== TASK 2: ECC IMPLEMENTATION =====

Elliptic Curve: $y^2 = x^3 + 2x + 3 \pmod{97}$
Generator Point $G = (3, 6)$

Private Key: 7
Public Key: (80, 10)

Original Message: 20

Encrypted Cipher:
 $C1 = (80, 87)$
 $C2 = 23$

Decrypted Message: 20